

DOCUMENTAÇÃO ESCRITA — SQUAD 16

Versão 2.0 — Revisada e Consolidada

1. Análise do Problema (Visão de Produto)

A Cencosud precisa de um simulador estratégico de operação de loja onde squads tomem decisões reais sobre preço, estoque, equipe e serviços, observando como cada escolha impacta o resultado financeiro e a posição competitiva em um jogo estruturado por rodadas.

Cada squad configura uma loja com as seguintes variáveis de decisão:

- Custo de compra (purchasePrice — definido globalmente no produto)
- Preço de venda (salePrice — configurável por rodada)
- Volume de vendas planejado (salesVolume — limitado pelo estoque disponível)
- Despesas fixas e variáveis (fixedExpenses, variableExpenses)
- Estoque (Inventory — controla o teto de salesVolume efetivo)

Essas variáveis alimentam o motor de cálculo financeiro, que gera um mini DRE por rodada. O sistema possui 3 camadas conceituais:

- 1. Gestão da Loja — cadastro, estoque e configuração
- 2. Motor de Simulação Financeira — cálculo do DRE
- 3. Competição entre Squads — ranking baseado em resultado financeiro

2. Glossário de Papéis (Roles)

■ v1 usava dois sistemas de papéis paralelos sem mapeamento entre si: Jornada do Usuário definia Gestor/Operador; Histórias de Usuário definia Amador/Profissional/Game Master. Isso gerava ambiguidade na implementação do controle de acesso.

✓ CORREÇÃO: Glossário único definido abaixo. A distinção Amador/Profissional é de UX/experiência, não de role no banco.

Role (banco)	Nome UX	Permissões principais
GAME_MASTER	Game Master / Gestor	Criar/encerrar rodadas, ajustar parâmetros globais, gerenciar squads e usuários, visualizar todos os resultados
PLAYER	Jogador (Amador ou Profissional)	Configurar RoundConfig da própria loja, visualizar DRE próprio e ranking geral
OBSERVER	Observador	Somente leitura — sem permissão de escrita

3. Entidades Principais do Sistema

Lista consolidada de todas as entidades, alinhada com a modelagem v2 do banco.

User

id, name, email, password, role (UserRole), leader (Boolean), squadId, createdAt

Squad

id, name, createdAt, updatedAt

Store

id, name, initialCapital, squadId, createdAt

Product

id, name, purchasePrice, salePrice (padrão), createdAt

Inventory

id, storeId, productId, quantity, agingCategory

Round

id, number, startDate, endDate, status (RoundStatus)

RoundConfig

id, roundId, storeId, fixedExpenses, variableExpenses, submittedAt

RoundConfigItem [NOVO]

id, roundConfigId, productId, salePrice, salesVolume

FinancialResult

id, roundId, storeId, roundConfigId, grossRevenue, costs, expenses, grossProfit, netProfit, netMargin, calculatedAt

4. Lógica de Cálculo Financeiro

O motor financeiro opera sobre os dados de RoundConfigItem, consultando Inventory para determinar o volume efetivo de vendas. O cálculo é feito por produto e depois agregado na loja.

Volume efetivo (por produto)

Antes de calcular receita, o sistema valida o estoque:

```
effectiveSalesVolume = Math.min(item.salesVolume, inventory.quantity)
```

Receita Bruta

```
// Por item
```

```
itemGrossRevenue = item.salePrice * effectiveSalesVolume
```

```
// Total da loja na rodada
```

```
grossRevenue = sum(itemGrossRevenue) para todos os itens
```

Custos

```
// Por item – purchasePrice vem de Product
```

```
itemCost = product.purchasePrice * effectiveSalesVolume
```

```
// Total
```

```
costs = sum(itemCost) para todos os itens
```

Lucro Bruto

```
grossProfit = grossRevenue - costs
```

Lucro Líquido

```
netProfit = grossProfit - roundConfig.fixedExpenses - roundConfig.variableExpenses
```

Margem Líquida

■ v1 não tratava divisão por zero. Se grossRevenue = 0, o sistema lançaria exceção em produção.

✓ CORREÇÃO: Adicionada guarda contra divisão por zero.

```
netMargin = grossRevenue !== 0  
  
  ? (netProfit / grossRevenue) * 100  
  
  : 0
```

5. financeService.js — Versão Corrigida

Arquivo: src/services/financeService.js

```
/**  
 * Calcula o DRE completo de uma loja para uma rodada.  
 * @param {Object} roundConfig - { fixedExpenses, variableExpenses }  
 * @param {Array} items - [{ salePrice, salesVolume, product: { purchasePrice } }]  
 * @param {Array} inventory - [{ productId, quantity }]  
 * @returns {Object} DRE completo  
 */  
  
function calcularDRE(roundConfig, items, inventory) {  
  const inventoryMap = Object.fromEntries(  
    inventory.map(i => [i.productId, i.quantity])  
  );  
  
  let grossRevenue = 0;  
  let costs = 0;  
  
  for (const item of items) {  
    const availableStock = inventoryMap[item.productId] ?? 0;  
    const effectiveVolume = Math.min(item.salesVolume, availableStock);  
  
    grossRevenue += item.salePrice * effectiveVolume;  
    costs += item.product.purchasePrice * effectiveVolume;  
  }  
  
  const expenses = roundConfig.fixedExpenses + roundConfig.variableExpenses;  
  const grossProfit = grossRevenue - costs;  
  const netProfit = grossProfit - expenses;  
  const netMargin = grossRevenue !== 0  
    ? (netProfit / grossRevenue) * 100
```

```

: 0;

return { grossRevenue, costs, expenses, grossProfit, netProfit, netMargin };
}

module.exports = { calcularDRE };

```

6. Estrutura do Backend Node.js

Stack: Node.js + Express + Prisma + PostgreSQL

```

src/
  controllers/
    userController.js
    storeController.js
    simulationController.js
    roundController.js      // Novo – gerencia ciclo de vida das rodadas
    rankingController.js    // Novo – serve dados do ranking
  services/
    financeService.js       // Motor de cálculo – v2 com validação de estoque
    simulationService.js     // Orquestra financeService + persistência
    rankingService.js        // Calcula ranking on-demand via FinancialResult
  repositories/
    storeRepository.js
    inventoryRepository.js   // Novo – acesso ao estoque
    roundConfigRepository.js
    financialResultRepository.js
  routes/
    userRoutes.js
    storeRoutes.js
    simulationRoutes.js
    roundRoutes.js
    rankingRoutes.js
  middlewares/
    authMiddleware.js       // Valida JWT
    roleMiddleware.js        // Verifica role (GAME_MASTER, PLAYER, OBSERVER)
  utils/
    dreCalculator.js
  prisma/

```

schema.prisma

seed.js

7. Endpoints Principais da API

Método	Rota	Role mínimo	Descrição
POST	/auth/login	público	Autenticação — retorna JWT
GET	/users	GAME_MASTER	Lista usuários
POST	/users	GAME_MASTER	Cria usuário
PUT	/users/:id	GAME_MASTER	Atualiza usuário
DELETE	/users/:id	GAME_MASTER	Remove usuário (valida se é líder)
GET	/products	PLAYER	Lista produtos disponíveis
POST	/products	GAME_MASTER	Cria produto
POST	/rounds	GAME_MASTER	Cria nova rodada
PATCH	/rounds/:id/close	GAME_MASTER	Encerra rodada e dispara cálculo
POST	/rounds/:id/config	PLAYER	Submete RoundConfig da loja
POST	/simulation/preview	PLAYER	Simula DRE sem salvar (modo preview)
GET	/rounds/:id/results	PLAYER	Retorna FinancialResult da rodada
GET	/ranking	PLAYER	Retorna ranking calculado on-demand

8. Sistema de Ranking

■ v1 propunha ORDER BY `margemLiquida` em `Squad`, mas `Squad.score` era estático. Ranking não refletia evolução por rodada.

✓ CORREÇÃO: Ranking calculado on-demand via `FinancialResult`, agrupado por `squad` e rodada.

```
// rankingService.js

async function getRanking(roundId) {

  const results = await prisma.financialResult.findMany({

    where: { roundId },

    include: { store: { include: { squad: true } } },

    orderBy: { netMargin: 'desc' }

  });

  return results.map((r, index) => ({

    position: index + 1,

    squadName: r.store.squad.name,

    storeName: r.store.name,

    netProfit: r.netProfit,

    netMargin: r.netMargin,

    grossRevenue: r.grossRevenue,
```

```
}});
```

```
}
```

9. Seed de Produtos

```
// prisma/seed.js
```

```
const products = [
```

```
  { name: 'Arroz 5kg',    purchasePrice: 12.00, salePrice: 18.00 },
```

```
  { name: 'Feijão 1kg',   purchasePrice:  4.50, salePrice:  7.50 },
```

```
  { name: 'Macarrão 500g',purchasePrice:  2.80, salePrice:  4.90 },
```

```
  { name: 'Leite 1L',     purchasePrice:  3.20, salePrice:  5.50 },
```

```
  { name: 'Óleo 900ml',   purchasePrice:  5.00, salePrice:  8.00 },
```

```
];
```